

Unidad 2: Listas, tuplas, conjuntos y diccionarios en Python

Listas

Introducción a las Listas en Python

En Python, las listas son estructuras de datos muy versátiles que permiten almacenar una colección ordenada de elementos. A diferencia de otros lenguajes de programación, las listas en Python pueden contener elementos de diferentes tipos, incluyendo números, cadenas de texto e incluso otras listas.

Sintaxis básica

Para definir una lista en Python, se utilizan corchetes `[]` y los elementos de la lista se separan por comas. A continuación, se presentan algunos ejemplos de listas:

```
```python
Lista de números enteros
numeros = [1, 2, 3, 4, 5]

Lista de cadenas de texto
frutas = ["manzana", "banana", "cereza"]

Lista heterogénea (contiene diferentes tipos de datos)
mi_lista = [1, "Hola", 3.14, True]
```
```

Las listas son mutables, lo que significa que se pueden modificar después de haber sido creadas. Se pueden agregar, eliminar o cambiar elementos dentro de una lista.

Ejemplo de Listas en Python

Definición de Listas

A continuación, se muestran ejemplos prácticos de cómo definir y utilizar listas en Python.

```
```python

Definiendo una lista de números
numeros = [10, 20, 30, 40, 50]

Definiendo una lista de cadenas de texto
nombres = ["Alice", "Bob", "Charlie"]

Definiendo una lista heterogénea
datos = [25, "Python", 3.14, False]

```
```

Acceso por Índice

Los elementos de una lista se pueden acceder utilizando su índice. El índice comienza en 0 para el primer elemento. Veamos algunos ejemplos:

```
```python

Accediendo a elementos de la lista
primer_numero = numeros[0] # 10
segundo_nombre = nombres[1] # "Bob"

Accediendo a elementos de una lista heterogénea
primer_dato = datos[0] # 25
segundo_dato = datos[1] # "Python"

```
```

Slicing (Subconjuntos de Listas)

Python permite extraer subconjuntos de una lista utilizando la técnica de slicing. Esto se hace utilizando el operador `:` dentro de los corchetes.

```
```python

Definiendo una lista de números
numeros = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

Slicing: obteniendo los primeros cinco elementos
primeros_cinco = numeros[0:5] # [0, 1, 2, 3, 4]

Slicing: obteniendo elementos desde el índice 5 hasta el final
```

```
desde_cinco = numeros[5:] # [5, 6, 7, 8, 9]

Slicing: obteniendo un subconjunto de elementos
sub_lista = numeros[2:7] # [2, 3, 4, 5, 6]

Slicing con el tercer parámetro (paso):
Obteniendo elementos desde el índice 0 hasta el 9 con un paso de 2
pares = numeros[0:10:2] # [0, 2, 4, 6, 8]

'''
```

---

## Operaciones Básicas

Algunas operaciones básicas que se pueden realizar con listas incluyen agregar, eliminar y modificar elementos.

```
```python

# Lista inicial
mi_lista = [1, 2, 3]

# Agregar un elemento al final de la lista
mi_lista.append(4) # [1, 2, 3, 4]

# Insertar un elemento en una posición específica
mi_lista.insert(1, 1.5) # [1, 1.5, 2, 3, 4]

# Eliminar un elemento específico
mi_lista.remove(1.5) # [1, 2, 3, 4]

# Modificar un elemento existente
mi_lista[0] = 5 # [5, 2, 3, 4]

'''
```

Las listas en Python son una herramienta poderosa y flexible para trabajar con colecciones de datos. Su capacidad para almacenar diferentes tipos de datos y las múltiples operaciones que se pueden realizar con ellas las hacen esenciales para cualquier desarrollador Python.

Tuplas

En Python, las tuplas son estructuras de datos que, al igual que las listas, permiten almacenar una colección ordenada de elementos. Sin embargo, a diferencia de las listas, las tuplas son inmutables, lo que significa que una vez creadas, sus elementos no pueden

ser modificados. Esta característica hace que las tuplas sean útiles para almacenar datos que no deben cambiar a lo largo del programa.

Características de las Tuplas

1. **Inmutabilidad:** Una vez definida, no se pueden cambiar los elementos de una tupla.
2. **Definición:** Se definen utilizando paréntesis `()` en lugar de corchetes `[]`.
3. **Acceso Rápido:** Al ser inmutables, Python puede optimizar el acceso a sus elementos, lo que las hace ligeramente más rápidas que las listas para algunas operaciones.
4. **Heterogeneidad:** Al igual que las listas, las tuplas pueden contener elementos de diferentes tipos.

A continuación se muestra la sintaxis básica para definir una tupla:

```
```python
Definiendo una tupla
mi_tupla = (1, 2, 3, 4, 5)
```
```

Ejemplo de Tuplas

Definición de Tuplas

Veamos algunos ejemplos prácticos de cómo definir y utilizar tuplas en Python:

```
```python
Tupla de números enteros
numeros = (10, 20, 30, 40, 50)

Tupla de cadenas de texto
nombres = ("Alice", "Bob", "Charlie")

Tupla heterogénea (diferentes tipos de datos)
datos = (25, "Python", 3.14, False)
```
```

Acceso por Índice

Los elementos de una tupla se pueden acceder utilizando su índice, de forma similar a las listas:

```
```python

Accediendo a elementos de la tupla
primer_numero = numeros[0] # 10
segundo_nombre = nombres[1] # "Bob"

Accediendo a elementos de una tupla heterogénea
primer_dato = datos[0] # 25
segundo_dato = datos[1] # "Python"

```
```

Slicing (Subconjuntos de Tuplas)

También se puede aplicar slicing a las tuplas para obtener subconjuntos de elementos:

```
```python

Definiendo una tupla de números
numeros = (0, 1, 2, 3, 4, 5, 6, 7, 8, 9)

Slicing: obteniendo los primeros cinco elementos
primeros_cinco = numeros[0:5] # (0, 1, 2, 3, 4)

Slicing: obteniendo elementos desde el índice 5 hasta el final
desde_cinco = numeros[5:] # (5, 6, 7, 8, 9)

Slicing: obteniendo un subconjunto de elementos
sub_tupla = numeros[2:7] # (2, 3, 4, 5, 6)

```
```

Listas y Tuplas

Similitudes

1. **Ordenadas:** Tanto las listas como las tuplas mantienen el orden de los elementos.
2. **Acceso por Índice:** Se puede acceder a los elementos usando índices numéricos.
3. **Heterogeneidad:** Ambas pueden contener elementos de diferentes tipos de datos.

Diferencias

1. Mutabilidad:

- **Listas:** Son mutables, lo que significa que sus elementos pueden ser cambiados, agregados o eliminados.
- **Tuplas:** Son inmutables, una vez creadas, sus elementos no pueden ser modificados.

2. Definición:

- **Listas:** Se definen usando corchetes `[]`.
- **Tuplas:** Se definen usando paréntesis `()`.

3. Uso:

- **Listas:** Se utilizan cuando se espera que los datos cambien a lo largo del programa.
- **Tuplas:** Se utilizan cuando se necesita un conjunto de datos que no cambie.

4. **Rendimiento:** Las tuplas pueden ser más rápidas que las listas para algunas operaciones debido a su inmutabilidad.

Ejemplo Comparativo

```
```python
```

```
Lista
```

```
mi_lista = [1, 2, 3]
```

```
mi_lista.append(4) # [1, 2, 3, 4]
```

```
Tupla
```

```
mi_tupla = (1, 2, 3)
```

```
mi_tupla.append(4) # Esto daría un error porque las tuplas son inmutables,
```

```
de hecho, no existe el método append en las tuplas
```

```
```
```

Las listas y las tuplas son fundamentales en Python y elegir cuál usar dependerá de la necesidad de mutabilidad en los datos.

Conjuntos

Introducción a los Conjuntos y Diccionarios en Python

En Python, los conjuntos y diccionarios son otro tipo de colecciones que, al igual que las tuplas y listas, permiten almacenar múltiples elementos en una sola variable. Aunque ambos se utilizan para agrupar datos, tienen características y usos distintos que los diferencian de otros tipos de colecciones como las listas y las tuplas.

Diferencias y Similitudes

1. Estructura:

- **Conjuntos** Son colecciones no ordenadas de elementos únicos. No permiten duplicados y no mantienen el orden de los elementos.
- **Diccionarios**: Son colecciones de pares clave-valor. Cada clave es única y se utiliza para almacenar y acceder a los valores correspondientes.

2. Mutabilidad:

- **Conjuntos**: Son mutables, es decir, se pueden agregar o eliminar elementos después de su creación.
- **Diccionarios**: También son mutables, permitiendo la modificación de pares clave-valor existentes, así como la adición y eliminación de nuevos pares.

3. Acceso a Elementos:

- **Conjuntos**: No permiten el acceso a elementos mediante índices ya que no mantienen un orden.
- **Diccionarios**: Permiten el acceso rápido a los valores mediante sus claves únicas.

4. Usos Comunes:

- **Conjuntos**: Se utilizan para almacenar elementos únicos y realizar operaciones matemáticas como unión, intersección y diferencia.
 - **Diccionarios**: Se utilizan para almacenar datos en pares clave-valor, permitiendo búsquedas rápidas y eficientes.
-

¿Qué son los Conjuntos?

Los conjuntos en Python son colecciones no ordenadas de elementos únicos. Esto significa que no pueden contener elementos duplicados y que el orden de los elementos no está garantizado.

Sintaxis Básica

Para definir un conjunto en Python, se utilizan llaves `{}` o la función `set()`. A continuación, se presentan algunos ejemplos de cómo definir y utilizar conjuntos:

```
```python

Definiendo un conjunto de números
numeros = {1, 2, 3, 4, 5}

Usando la función set()
letras = set(['a', 'b', 'c', 'd'])

Conjunto vacío
conjunto_vacio = set()

```
```

Características de los Conjuntos

1. **Elementos Únicos:** Los conjuntos no permiten elementos duplicados. Si se intenta agregar un duplicado, será ignorado.

```
```python

numeros = {1, 2, 2, 3}
print(numeros) # Resultado: {1, 2, 3}

...`
```

2. **No Ordenados:** Los elementos en un conjunto no tienen un orden específico. No se puede acceder a ellos mediante índices.

3. **Operaciones de Conjuntos:** Python proporciona varias operaciones para trabajar con conjuntos, como la unión, intersección y diferencia.

---

## Ejemplos de Operaciones con Conjuntos

- **Unión:** Combina todos los elementos de dos conjuntos, eliminando duplicados.

```
```python

conjunto1 = {1, 2, 3}
conjunto2 = {3, 4, 5}
union = conjunto1 | conjunto2 # Resultado: {1, 2, 3, 4, 5}

...`
```

- **Intersección:** Devuelve los elementos que están presentes en ambos conjuntos.

```
```python

interseccion = conjunto1 & conjunto2 # Resultado: {3}

...`
```

- **Diferencia:** Devuelve los elementos que están en el primer conjunto pero no en el segundo.

```
```python

diferencia = conjunto1 - conjunto2 # Resultado: {1, 2}

...`
```

Uso de Métodos en Conjuntos

Además de las operaciones básicas, los conjuntos en Python tienen varios métodos útiles:

- ``add()``: Agrega un elemento al conjunto.

```
```python
numeros.add(6) # numeros ahora es {1, 2, 3, 4, 5, 6}
```
```

- ``remove()``: Elimina un elemento del conjunto. Genera un error si el elemento no existe.

```
```python
numeros.remove(6) # numeros ahora es {1, 2, 3, 4, 5}
```
```

- ``discard()``: Elimina un elemento del conjunto si está presente. No genera error si el elemento no existe.

```
```python
numeros.discard(6) # No genera error
```
```

- ``clear()``: Elimina todos los elementos del conjunto.

```
```python
numeros.clear() # numeros ahora es un conjunto vacío
```
```

Los conjuntos son especialmente útiles cuando se necesita trabajar con datos únicos y realizar operaciones matemáticas eficientes.

Diccionarios

¿Qué son los Diccionarios?

Los diccionarios en Python son estructuras de datos que permiten almacenar colecciones de pares clave-valor. A diferencia de las listas y tuplas, que son secuencias de elementos, los diccionarios son colecciones no ordenadas, lo que significa que no mantienen el orden de inserción de los elementos. Cada clave en un diccionario es única y se utiliza para acceder a su valor correspondiente de manera rápida y eficiente.

Sintaxis Básica

Para definir un diccionario en Python, se utilizan llaves `{}` y los pares clave-valor se separan por comas, con la clave y el valor separados por dos puntos `:`. A continuación, se presentan algunos ejemplos de cómo definir y utilizar diccionarios:

```
```python

Definiendo un diccionario con claves y valores
persona = {
 "nombre": "Juan",
 "edad": 30,
 "ciudad": "Madrid"
}

Otro ejemplo de diccionario
inventario = {
 "manzanas": 10,
 "naranjas": 5,
 "peras": 7
}

```
```

Características de los Diccionarios

1. **Colecciones No Ordenadas:** Los diccionarios no mantienen el orden de los elementos. En Python 3.7 y versiones posteriores, los diccionarios mantienen el orden de inserción de los elementos, pero esto no es algo en lo que se deba confiar siempre para versiones anteriores.

2. **Pares Clave-Valor:** Cada elemento en un diccionario está formado por dos partes: una clave y un valor. Las claves deben ser únicas e inmutables (pueden ser cadenas, números o tuplas), mientras que los valores pueden ser de cualquier tipo.

3. **Acceso Rápido a Valores:** Se puede acceder a los valores en un diccionario de manera eficiente utilizando sus claves.

Ejemplos Prácticos

Definición y Acceso

Para definir un diccionario y acceder a sus valores, se utilizan las claves correspondientes:

```
```python

Definiendo un diccionario
alumno = {
 "nombre": "Ana",
 "edad": 22,
 "curso": "Matemáticas"
}

Accediendo a valores utilizando claves
nombre_alumno = alumno["nombre"] # "Ana"
edad_alumno = alumno["edad"] # 22

```
```

Modificación de Valores

Los diccionarios son mutables, por lo que se pueden modificar los valores existentes, agregar nuevos pares clave-valor y eliminar elementos:

```
```python

Modificando un valor existente
alumno["edad"] = 23 # La edad ahora es 23

Agregando un nuevo par clave-valor
alumno["ciudad"] = "Barcelona" # Se añade la clave "ciudad" con el valor "Barcelona"

Eliminando un par clave-valor
del alumno["curso"] # Se elimina la clave "curso" y su valor

```
```

Métodos Útiles

Python proporciona varios métodos útiles para trabajar con diccionarios:

- **`keys()`**: Devuelve una vista de las claves del diccionario.

```
```python
claves = alumno.keys() # dict_keys(['nombre', 'edad', 'ciudad'])
```
```

- **`values()`**: Devuelve una vista de los valores del diccionario.

```
```python
valores = alumno.values() # dict_values(['Ana', 23, 'Barcelona'])
```
```

- **`items()`**: Devuelve una vista de los pares clave-valor del diccionario.

```
```python
items = alumno.items() # dict_items([('nombre', 'Ana'), ('edad', 23), ('ciudad', 'Barcelona')])
```
```

- **`get()`**: Devuelve el valor para una clave dada, o `None` si la clave no existe.

```
```python
curso = alumno.get("curso") # None, ya que la clave "curso" fue eliminada
```
```

- **`update()`**: Actualiza el diccionario con los pares clave-valor de otro diccionario o con pares clave-valor proporcionados como argumentos.

```
```python
alumno.update({"edad": 24, "curso": "Física"}) # Actualiza la edad y añade el curso
```
```

Los diccionarios son extremadamente útiles cuando se necesita una asociación rápida entre claves y valores, lo que permite búsquedas eficientes y manipulaciones flexibles de datos.

Material complementario:

Te dejamos material ampliado de la clase:

- [Listas en Python](#): Las listas son estructuras de datos que permiten almacenar colecciones ordenadas de elementos que pueden ser de diferentes tipos y son mutables.
- **Tuplas en Python**: Las tuplas son similares a las listas, pero son inmutables, es decir, una vez creadas, sus elementos no pueden cambiarse. Puedes aprender más sobre ellas en [este artículo](#).
- [Conjuntos \(Sets\) y sus funciones](#): Los conjuntos son colecciones desordenadas de elementos únicos.
- [Diccionarios y recursos](#): Los diccionarios son colecciones desordenadas de pares clave-valor.